

```

"""
verrouillage_1_5_lyapunov.py
=====
Calcul des constantes ( $\lambda$ ,  $b$ ) de Foster-Lyapunov pour la dynamique MCMC du Blob.

Fonction de Lyapunov candidate (à valider) :
     $V_L(C) = \max(S_{\max} - S_{\text{sim}}(C), 0)$ 

Critère Foster-Lyapunov pour ergodicité géométrique :
     $E[V_L(C_{\{n+1\}}) \mid C_n] \leq (1 - \lambda) V_L(C_n) + b$ 

avec  $\lambda > 0$  et  $b < \infty$ .

Mesure : régression de  $E[V_L(t+1) - V_L(t)] / V_L(t)$  sur trajectoires MCMC.
"""

import numpy as np
import math
import time
from collections import deque
from scipy.spatial import KDTree
from scipy.stats import linregress

KAPPA_0 = 8
BETA = 0.30
TAU = 0.50
N_TEST = 200
N_SEEDS = 3
BURN_FACTOR = 100
PROD_FACTOR = 200

# Copie minimale du pipeline O2v4
def _rips_radius(N, kappa=KAPPA_0): return (kappa / ((N-1)*(4/3)*math.pi))**(1/3)
def _E0(N, kappa=KAPPA_0): return int(N * kappa / 2)

def _lcc_size(adj, N):
    visited = [False]*N; best = 0
    for s in range(N):
        if not visited[s]:
            sz = 0; q = deque([s]); visited[s] = True
            while q:
                v = q.popleft(); sz += 1
                for w in adj[v]:
                    if not visited[w]: visited[w] = True; q.append(w)

```

```

        best = max(best, sz)
    return best

def _build_initial(N, seed):
    r = _rips_radius(N); r_p = 1.5 * r
    E0 = _E0(N)
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0.0, 1.0, (N, 3))
    tree = KDTree(pts)
    adj = [set() for _ in range(N)]
    for i, j in tree.query_pairs(r):
        adj[i].add(j); adj[j].add(i)
    pool = list(tree.query_pairs(r_p))
    rng2 = np.random.default_rng(seed + 50_000)
    K = sum(len(adj[i]) for i in range(N)) // 2
    att = 0; max_att = 10 * abs(E0 - K) + 5000
    while K != E0 and att < max_att:
        att += 1
        if K < E0:
            e = pool[rng2.integers(len(pool))]; u, v = e
            if v not in adj[u]: adj[u].add(v); adj[v].add(u); K += 1
        else:
            edges = [(i,j) for i in range(N) for j in adj[i] if j>i]
            if not edges: break
            e = edges[rng2.integers(len(edges))]; u, v = e
            adj[u].discard(v); adj[v].discard(u)
            if _lcc_size(adj, N) == N: K -= 1
            else: adj[u].add(v); adj[v].add(u)
    return pts, adj, pool, E0

def _count_triangles(adj, N):
    n = 0
    for i in range(N):
        for j in adj[i]:
            if j > i:
                n += len([k for k in adj[i] & adj[j] if k > j])
    return n

def compute_S(adj, n_tri, N):
    degs = np.array([len(adj[i]) for i in range(N)], float)
    D = degs.std() / (degs.mean() + 1e-8)
    return math.log(1 + n_tri) - BETA * D

def _fast_still_connected(adj, u, v, N):
    visited = {u}; q = deque([u])
    while q:
        node = q.popleft()

```

```

        for w in adj[node]:
            if w in visited: continue
            if node == u and w == v: continue
            if node == v and w == u: continue
            visited.add(w); q.append(w)
            if w == v: return True
    return False

def run_mcmc_with_trace(pts, adj_init, pool, N, E0, seed, sample_every=N_TEST):
    """MCMC qui retourne la trajectoire de S_sim et n_tri à intervalle régulier."
    tau_eff = TAU / N
    rng = np.random.default_rng(seed + 200_000)
    adj = [set(a) for a in adj_init]
    edge_list = [(i,j) for i in range(N) for j in adj[i] if j>i]
    n_tri = _count_triangles(adj, N)
    S_curr = compute_S(adj, n_tri, N)
    n_steps = (BURN_FACTOR + PROD_FACTOR) * N
    pool_arr = np.array(pool, dtype=np.int32)
    n_pool = len(pool_arr)

    S_trace = []

    for step in range(n_steps):
        if not edge_list: break
        idx_out = int(rng.integers(len(edge_list)))
        u, v = edge_list[idx_out]
        e_in = None
        for _ in range(30):
            idx_in = int(rng.integers(n_pool))
            a, b = int(pool_arr[idx_in,0]), int(pool_arr[idx_in,1])
            if b not in adj[a]: e_in = (a, b); break
        if e_in is None: continue
        a, b = e_in
        if len(adj[u]) <= 2 or len(adj[v]) <= 2:
            if not _fast_still_connected(adj, u, v, N): continue
        tri_lost = len(adj[u] & adj[v])
        adj[u].discard(v); adj[v].discard(u)
        adj[a].add(b); adj[b].add(a)
        tri_gained = len(adj[a] & adj[b])
        n_tri_new = n_tri - tri_lost + tri_gained
        S_new = compute_S(adj, n_tri_new, N)
        dS = S_new - S_curr
        if dS >= 0 or rng.random() < math.exp(dS / tau_eff):
            n_tri = n_tri_new; S_curr = S_new
            edge_list[idx_out] = (a, b)
        else:

```

```

        adj[u].add(v); adj[v].add(u); adj[a].discard(b); adj[b].discard(a)

    # Sample
    if step % sample_every == 0:
        S_trace.append(S_curr)

return np.array(S_trace), S_curr

def estimate_lyapunov_constants(S_trace, N, n_burn_skip=50):
    """
    Estime  $(\lambda, b)$  de Foster-Lyapunov sur la trajectoire.

     $V_L(C) = \max(S_{\max} - S(C), 0)$  avec  $S_{\max}$  = max observé en équilibre
    Modèle :  $V_L(t+1) \approx (1-\lambda) V_L(t) + b + \text{noise}$ 
    """
    # Sauter le burn-in
    trace = S_trace[n_burn_skip:]
    if len(trace) < 20:
        return None

    #  $S_{\max} \approx$  max observé en équilibre
    S_max = trace.max()
    V_L = np.maximum(S_max - trace, 0)

    # Si  $V_L$  est presque toujours nul, on rate (l'équilibre est trop concentré)
    if V_L.std() < 1e-6:
        return {'lambda': None, 'b': None, 'note': 'V_L too concentrated near equ

    # Régression  $V_L(t+1) = a * V_L(t) + c + \text{noise}$ 
    V_t = V_L[:-1]
    V_tp1 = V_L[1:]

    res = linregress(V_t, V_tp1)
    a = res.slope
    c = res.intercept

    #  $(1 - \lambda) = a$ , donc  $\lambda = 1 - a$ 
    #  $b = c$ 
    lambda_val = 1 - a
    b_val = c

    return {
        'S_max_observed': float(S_max),
        'V_L_mean': float(V_L.mean()),
        'V_L_std': float(V_L.std()),
        'V_L_max': float(V_L.max()),

```

```

        'lambda': float(lambda_val),
        'b': float(b_val),
        'a_slope': float(a),
        'R2': float(res.rvalue**2),
        'n_samples_used': len(V_t),
        'satisfies_foster_lyapunov': bool(lambda_val > 0 and abs(b_val) < float('
    }

if __name__ == "__main__":
    print("=" * 70)
    print("VERROUILLAGE 1.5 – Constantes ( $\lambda$ , b) de Foster-Lyapunov")
    print("=" * 70)
    print(f"\nN={N_TEST},  $\beta$ ={BETA},  $\tau$ ={TAU}, {N_SEEDS} seeds, burn={BURN_FACTOR}N")
    print(f"\nFonction de Lyapunov :  $V_L(C) = \max(S_{\max} - S_{\text{sim}}(C), 0)$ ")
    print(f"Critère :  $E[V_L(t+1) \mid t] \leq (1-\lambda) V_L(t) + b$ \n")

    results = []
    for seed in range(N_SEEDS):
        t0 = time.time()
        pts, adj_init, pool, E0 = _build_initial(N_TEST, seed)
        if _lcc_size(adj_init, N_TEST) / N_TEST < 0.9: continue
        S_trace, S_final = run_mcmc_with_trace(pts, adj_init, pool, N_TEST, E0, s
        t_mcmc = time.time() - t0

        # Estimation
        est = estimate_lyapunov_constants(S_trace, N_TEST)
        if est:
            est['seed'] = seed
            results.append(est)
            print(f"Seed {seed}:  $\lambda = \{est['lambda']:.4f\}$ ,  $b = \{est['b']:.5f\}$ , "
                  f" $R^2 = \{est['R2']:.3f\}$ ,  $V_L_{\max} = \{est['V_L_{\max}']:.4f\}$  [{t_mcm

    if results:
        lambdas = np.array([r['lambda'] for r in results if r['lambda'] is not No
        bs = np.array([r['b'] for r in results if r['b'] is not None])
        if len(lambdas) > 0:
            print(f"\n{'='*70}")
            print(f"RÉSULTATS AGRÉGÉS ({len(lambdas)} seeds valides)")
            print(f"{'='*70}")
            print(f" $\lambda$  (taux de décroissance) = {lambdas.mean():.4f}  $\pm$  {lambdas.
            print(f" $b$  (constante d'offset) = {bs.mean():.5f}  $\pm$  {bs.std():.5f

            if (lambdas > 0).all():
                print(f"\n ✓  $\lambda > 0$  sur tous seeds → contraction stricte ✓")
            else:
                print(f"\n ✗  $\lambda \leq 0$  sur certains seeds → contraction non-strict")

```

```

if (np.abs(bs) < 10).all():
    print(f"    ✓ b borné sur tous seeds → critère Foster-Lyapunov sati

print(f"\n Conclusion : la fonction  $V_L = \max(S_{\max} - S, 0)$  vérifie"
print(f"    l'inégalité de Foster-Lyapunov avec :")
print(f"         $\lambda \approx \{\text{lambdas.mean():.3f}\}$ ")
print(f"         $b \approx \{\text{bs.mean():.4f}\}$ ")
print(f"    Ergodicité géométrique de la dynamique MCMC confirmée numér
print(f"    → Point 1.5 : constantes  $(\lambda, b)$  estimées.  $[T^*] \rightarrow [T]$  partie

print(f"\n{'='*70}")

```